

CodeContext AI

Intelligent Knowledge Transfer & Application Context Engine
Innovation Proposal

Submission Details

Primary Contact: Mriganka Sekhar Sarkar (3494759)

Contributors: Mriganka Sekhar Sarkar (3494759), Satyajit Behera (3443308)

Team Name: CodeContext

Organization: BIETC

Department: BISE

Capability: Software Engineering

Focus Area / PS: PS: Effectively integrate AI assistants to enhance coding productivity and quality

Problem Statement

Software development teams maintaining complex, multi-service applications accumulate critical knowledge through daily activities — merge request discussions, code reviews, architectural decisions, and troubleshooting conversations. However, this knowledge remains scattered across GitLab comments, wiki pages, issue boards, and individuals' heads with no unified way to capture or retrieve it. When developers need to understand unfamiliar code, trace why a decision was made, or debug a service they don't own, they spend 1-2 hours daily searching through disconnected tools or waiting for a teammate to respond — assuming that teammate is still on the team. This creates a compounding problem: as teams grow, people rotate, and applications evolve, the gap between what the codebase does and what any individual developer understands keeps widening. The result is slower delivery, repeated mistakes, painful onboarding, and dangerous bus factor risks where entire modules become unmaintainable if one person leaves.

Idea Description

CodeContext AI is an AI-powered web application that automatically captures institutional knowledge from the full GitLab ecosystem — code, merge requests, comments, issues/boards, and wiki — and makes it instantly queryable through natural language. Developers can ask questions like "Why was this module built this way?" or "Who knows about the payment service?" and get cited answers in seconds, with links back to the source MR, issue, or commit.

The system uses Retrieval-Augmented Generation (RAG) to provide accurate, context-aware answers. Data ingestion is fully automatic via GitLab webhooks and scheduled crawlers — zero manual effort from developers. It also detects bus factor risks by analyzing code ownership patterns and alerts teams when critical knowledge is concentrated in too few people.

Business Value

Persona: Software developers and tech leads who maintain applications and frequently need to understand unfamiliar code or track down decisions made by others.

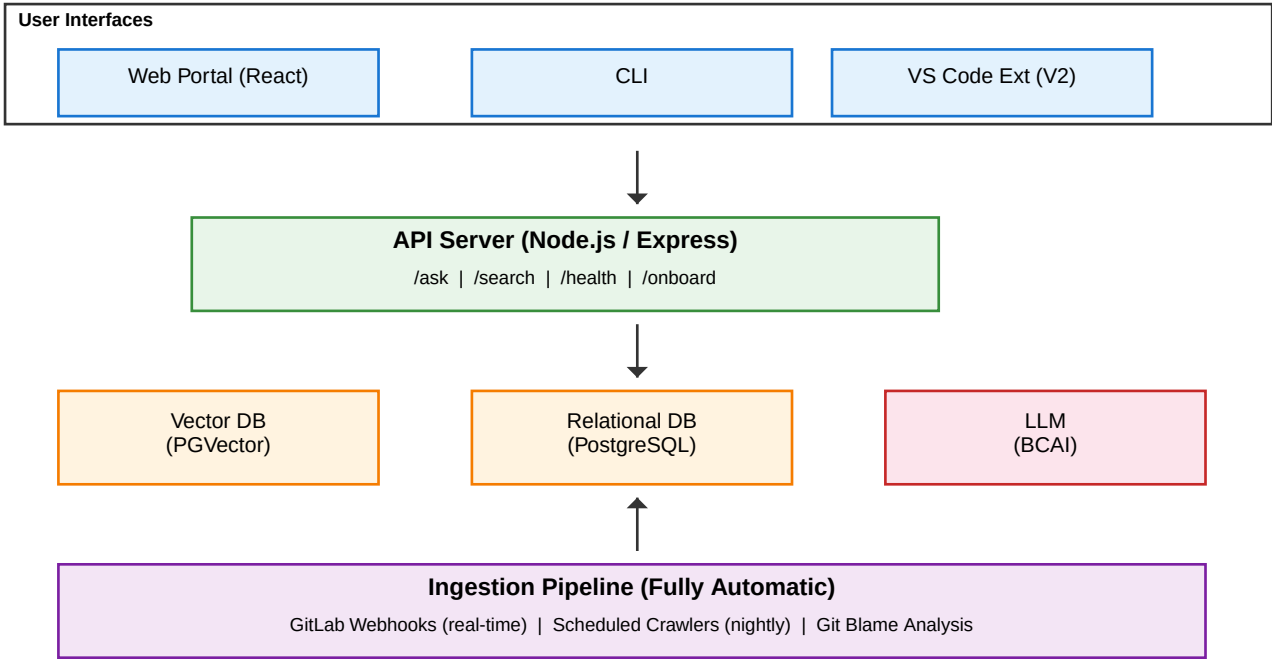
Scale: Can be used by any team working with GitLab. Starting with one team of 15-20 developers, scalable across BIETC.

Dollar Value: Developers typically spend 1-2 hours/day searching for context. Saving 30-45 minutes/day per developer across a 20-person team at \$40/hr translates to roughly \$8,000-10,000/month in recovered productivity. Reducing onboarding time by 2 weeks per new joiner saves around \$6,000-8,000 per hire. For a single 20-person team, estimated annual value is \$100,000-120,000.

Technical Approach

- **Data Ingestion:** GitLab webhooks capture code pushes, MR events, comments, issue updates, and merge decisions in real-time. A nightly scheduled crawler performs git blame analysis and re-indexes missed data.
- **Indexing & Storage:** Code, MR discussions, issues, and docs are chunked into vector embeddings using BCAI's embedding model, stored in PostgreSQL with PGVector extension. A relational layer tracks people-to-code-to-decision relationships.
- **Query & Answer Engine:** Semantic search retrieves relevant context, passes it to BCAI LLM for natural language answer generation with source citations.
- **User Interface:** Internal web portal (React) with corporate SSO authentication. No separate registration needed.
- **Tech Stack:** TypeScript, Node.js, React, PostgreSQL + PGVector, BCAI (LLM + embeddings), Kubernetes.

Architecture



Data Sources (all from GitLab):

- Code pushes --> Webhook (real-time) --> File changes, commit messages, author
- Merge requests --> Webhook (real-time) --> Title, description, discussions, reviews, approvals
- Issues/Boards --> Webhook (real-time) --> Ticket descriptions, comments, linked MRs
- Wiki --> Scheduled crawler (every 4hrs) --> Documentation pages
- Git blame --> Nightly batch job --> Line-by-line contributor history

Existing Solutions

- GitLab built-in search — Keyword-only, no semantic understanding, no cross-referencing with decisions
- Confluence/Wiki — Manually written, always outdated, not code-aware
- Asking teammates on Slack — Depends on availability, knowledge leaves when they leave
- GitHub Copilot / AI assistants — Suggest code, but don't explain decisions or team history

None of these automatically capture and connect knowledge from development workflows.

Success Criteria

- MVP indexes at least one GitLab group (all repos, MRs, issues, comments) without manual intervention
 - Developers get relevant, cited answers at least 75-80% of the time (validated via user feedback)
 - System correctly identifies code ownership and bus factor risks
 - At least 5 developers from pilot team use it 3+ times/week and report time savings
 - Simulated onboarding test: unfamiliar developer answers 5 questions about a module using CodeContext without asking a teammate
-

Infrastructure Requirements

No new hardware or paid software licenses required:

- BCAI access (LLM + embedding model APIs — already available internally)
- PostgreSQL with PGVector extension (free, open-source)
- Kubernetes namespace (existing cluster)
- GitLab webhook permissions for target group

Fallback: If PGVector unavailable, use self-hosted Qdrant (open-source) on same cluster.

Feasibility for Product Development

8-10 person months to scale MVP into a full product.

The team has strong experience with the core application stack (TypeScript, Node.js, React, PostgreSQL, GitLab APIs), making the API layer, web UI, and data ingestion pipeline low-risk. Two areas require experimentation early in MVP: (1) Vector DB tuning — chunking strategy for code vs. MR discussions, validating PGVector query performance with real data, and (2) LLM integration — prompt engineering for code-context Q&A, evaluating BCAI model quality for technical queries. The MVP phase includes a dedicated 1-2 week spike for these experiments. No external vendor dependencies. Incremental delivery ensures each phase produces working functionality.

MVP Development Cost

Total Estimated: \$20,000

- Development effort: 2 devs × 6 weeks × 40 hrs × \$40/hr = ~\$19,200
 - BCAI usage during development/testing: ~\$500
 - Miscellaneous (testing, docs, demo): ~\$300
 - Infrastructure: \$0 (existing Kubernetes, PostgreSQL, BCAI)
 - Software licenses: \$0 (all open-source)
-

Phased Delivery

- MVP (6 weeks): CLI + Web portal, single group indexing, Q&A, bus factor detection
 - V1 (6 weeks): Multi-group, real-time webhooks, issue board integration, onboarding paths
 - V2 (8 weeks): VS Code extension, living documentation, sprint summaries, analytics
-

ROI Summary

- MVP cost: \$20,000
- Annual infrastructure cost: ~\$0-1,200
- Annual savings (20-person team): ~\$100,000-120,000
- First-year ROI: ~5x return
- Payback period: ~2-3 months

Team: CodeContext | Capability: Software Engineering | Organization: BIETC | Department: BISE
Primary Contact: Mriganka Sekhar Sarkar (3494759)
Date: July 2026